

IDEA Platform — MC-Native Design

Status: Draft

Date: 2026-05-29

Author: Axle (with Koen)

Type: Platform redesign — replaces ad-hoc workflow decisions across multiple older docs

Supersedes: Relevant sections of [virtual-company-design.md](#) and [mc-deeper-integration.md](#) (closed)

Purpose

This document defines a target architecture for the IDEA virtual company platform that is maximally compatible with how OpenClaw and Mission Control are designed to work, based on the OpenClaw 2026.4.2 documentation and the decisions reached during the design session of 2026-05-29.

It is written to be executed faithfully by agents. Every design decision records the reasoning behind it. Migration steps are explicit and ordered.

Table of Contents

1. [Conceptual Foundation — Sessions](#)
 2. [Architecture Overview](#)
 3. [Work Cycle](#)
 4. [OpenClaw Configuration](#)
 5. [Installation — Clean Setup](#)
 6. [Agent Trigger Model](#)
 7. [Branch and Merge Workflow](#)
 8. [Graphify Integration](#)
 9. [Backup Strategy](#)
 10. [Skills Inventory](#)
 11. [MC Heartbeat — Online/Offline Status](#)
 12. [What Changes vs Current Setup](#)
 13. [Migration Runbook](#)
-

1. Conceptual Foundation — Sessions

Understanding sessions is prerequisite to understanding the rest of this design.

What a session is

A **session** is OpenClaw's unit of conversational continuity. It is a stored conversation transcript (a `.jsonl` file on disk) plus routing metadata. Every message an agent receives is routed into a session. The session accumulates a history. When the model runs, it sees that history — that is the agent's "working memory" for the current conversation.

Sessions are stored at:

```
~/ .openclaw/agents/<agentId>/sessions/<sessionId>.jsonl
```

Each **source** gets its own session: - Each Telegram group → its own isolated session - Cron jobs → a fresh isolated session per run (never shared with the Telegram session) - Spawned sub-agents → their own isolated session

The session is not the same as the agent. One agent has many sessions (one per bound group, plus transient cron sessions). The agent's persistent identity lives in workspace files (AGENTS.md, MEMORY.md, etc.), not in sessions.

What starts a new session

Trigger	Result
First message from a new source (new group, new DM)	New session created for that source
Daily reset fires (4:00 AM gateway local time, default)	New session ID — conversation history wiped
Idle reset fires (if configured)	New session after N minutes with no real user interaction
User sends <code>/new</code> or <code>/reset</code> in chat	Immediate fresh session
Cron job fires	Fresh isolated session per run (discarded after run)
Webhook fires	Isolated session per hook
<code>sessions_spawn</code> call	New isolated sub-agent session

What ends a session / causes the next message to open a new one

Cause	Notes
Daily reset clock fires	Even mid-conversation

Cause	Notes
Idle timeout fires	Based on last real user message only
<code>/new</code> or <code>/reset</code> command	User-initiated
Gateway restart (if reset condition met while offline)	Session may or may not resume depending on reset config

Critical: what does NOT extend idle freshness

Heartbeat wakeups, cron system events, and exec calls do **not** extend the idle timer. Only real user messages (from Telegram) do. An agent that receives only cron wakeups will eventually idle out regardless of how frequently the cron fires.

Session design decision for IDEA

Disable the daily reset. Use a 36-hour idle reset instead. Start new sessions manually with `/new`.

Rationale: the daily reset at 4 AM destroys conversation context at a fixed clock time, independent of what the agent is doing. It is useful for simple personal assistants that should "feel fresh" every day. For IDEA agents, continuity within a working thread matters more than daily freshness. Agents re-load their persistent context (AGENTS.md, MEMORY.md, GRAPH_REPORT.md) at every session start regardless — so losing the Telegram chat history is the only practical effect of the reset, and that is usually a cost rather than a benefit.

A 36-hour idle window is long enough to survive a full night plus a working day without breaking the session. Koen controls session boundaries explicitly: send `/new` or `/reset` in the Telegram group to start a fresh session at any point — before switching topics, before handing off a task to a sub-session, or whenever a clean context is desirable. The session also resets naturally after 36 hours of silence.

Starting tasks in a fresh session from Telegram: If you want to start a new session and immediately give it a task, send two messages: 1. `/new` — resets the session 2. Your task message — picked up in the fresh context

Or send a single message containing `/new` followed by your task on the next line. The agent treats the reset and the task as one turn.

Why not keep sessions open indefinitely and compact instead?

Context compaction (summarising the conversation history in-place to free up token space) is a valid rescue mechanism when a session runs long. Claude Code supports it explicitly via `/compact`. But compaction is lossy by nature — some information is always dropped in the summarisation —

and it does not address the deeper problem: context rot. Model performance degrades as context grows, because attention is spread across more tokens and older irrelevant content distracts from the current task. This degradation begins well before the token limit is reached, not at it.

More importantly, continuity across sessions does not require keeping a session alive. MEMORY.md, daily notes, and GRAPH_REPORT.md already serve as the inter-session bridge — curated, stable, and always loaded fresh at session start. This is equivalent to what Anthropic's long-running agent research calls a "progress file", except it is human-readable and intentionally maintained rather than auto-generated. Starting a new session with a clean context and a rich persistent memory is strictly better than continuing a stale one with compacted noise.

Target config:

```
{
  session: {
    reset: {
      idleMinutes: 2160 // 36 hours of inactivity opens a fresh
session; no nightly reset
    }
  }
}
```

The daily default reset is disabled by setting an explicit `idleMinutes` without a daily schedule. A 36-hour idle window means: sessions survive overnight and through the next working day without breaking. Koen starts new sessions manually with `/new` or `/reset` in Telegram whenever a clean context is wanted. Automatic resets happen only after 36 hours of true silence.

2. Architecture Overview

The three surfaces and what each is for

Surface	Purpose	Best for
Mission Control	Task management, work assignment, code review, approvals, audit trail	Work record, task state
Telegram	Real-time conversation, direction-setting, agent steering	Mobile, primary daily use

Surface	Purpose	Best for
OpenClaw Control UI	Browser chat with live tool streaming, config, session inspection	Desktop, watching agent work

All three serve different purposes and none replicates another. MC is the work record. Telegram is the primary conversation channel. The OpenClaw Control UI is a desktop supplement that adds visibility into what an agent is doing in real time.

Telegram and the Control UI talk to the **same underlying agent sessions** — a conversation started in Telegram continues in the Control UI and vice versa.

Component map

```

Koen (phone/laptop)
├─ Telegram → each agent's group → real-time conversation (primary, mobile)
├─ OpenClaw Control UI (Tailscale :18789) → desktop chat + live tool streaming
└─ MC UI (Tailscale :4000/:8000) → task boards → work assignment + review + approval

Pi (wizardly-hugle)
├─ OpenClaw gateway (native, systemd, pi user)
│   ├── Axle session (bound to IDEA-Axle Telegram group)
│   ├── Pixel session (bound to IDEA-Pixel Telegram group)
│   ├── Beacon session (bound to IDEA-Beacon Telegram group)
│   ├── Marco session (bound to IDEA-Marco Telegram group)
│   └─ Atlas session (bound to IDEA-Atlas Telegram group)
├─ Mission Control (Docker Compose)
│   ├── MC backend (FastAPI, port 8000)
│   ├── MC frontend (Next.js, port 4000)
│   ├── PostgreSQL 16 (task/board data)
│   └─ Redis (job queue)
└─ Graphify cache (sparse git clone, read-only, updated by cron)

Tailscale (external HTTPS access for Koen)
├─ OpenClaw Control UI → https://idea.tail2d60.ts.net:18789
├─ MC frontend → https://idea.tail2d60.ts.net:4000
└─ MC backend API → https://idea.tail2d60.ts.net:8000

```

(MC backend connects to OpenClaw gateway directly on Docker bridge – not via Tailscale)

OpenClaw Control UI — desktop chat surface

The OpenClaw Control UI at <https://idea.tail2d60.ts.net:18789> provides a browser-based chat panel for each agent. It is the best surface for:

- **Watching complex work happen** — tool calls stream live as the agent works; each tool invocation, file read, API call, and code execution is visible in real time. Telegram only shows the final response.
- **Desktop sessions** — when sitting at a computer and wanting a richer interface than Telegram.
- **Session inspection** — `/status`, `/context list`, and other session commands show structured output here.
- **Config changes** — the built-in config editor applies changes with hot-reload.

Telegram remains the primary channel for mobile access and daily interaction. Both surfaces share the same agent sessions — switching between them mid-conversation works seamlessly.

Access: Exposed via Tailscale Serve (Step 5 of install). Navigate to the URL above, or from the Pi: [openclaw dashboard](#).

Key architectural constraint: MC connects to OpenClaw directly

MC and OpenClaw both run on the same Pi. MC backend should connect to the OpenClaw gateway using the Pi's local address, not via Tailscale. Tailscale Serve is for external HTTPS access (Koen's browser); it is not the right path for internal service-to-service communication on the same machine.

This eliminates the socat proxy that the current setup uses (see §5).

3. Work Cycle

Standard task cycle

1. Koen creates task in MC
 - Sets board (agent's board), title, description, priority
 - Sets `assigned_agent_id` to the board's owning agent (always the same in 1:1 model)
 - Leaves task in "inbox"

2. Koen moves task to "in_progress"
 - Agent picks it up on next heartbeat poll (see §6)
 - OR Koen sends a Telegram message to make it immediate
3. Agent works
 - Works on a feature branch (not main)
 - Posts progress as MC task comments
 - Updates task to "review" when done
 - Posts branch name in task comment
4. Koen reviews
 - Reads task comment thread in MC
 - Reviews code diff on GitHub compare view:
`github.com/koenswings/<repo>/compare/main...<branch>`
 - No GitHub PR object created – no PR UI, no formal review process
5. Koen approves in MC
 - Moves task to "done" (or uses MC approval if configured)
 - This is the signal for the agent to merge
6. Agent merges to main
 - Executes: `git merge <branch> && git push origin main`
 - Posts merge confirmation as MC task comment
 - Graphify rebuild triggers automatically (see §8)

No additional approval loops

The MC task approval cycle is the only gate. There is no: - GitHub PR requirement - Extra Telegram confirmation step - BACKLOG.md file to maintain - Telegram command interface for task management

Telegram is for conversation. MC is for task state.

Session isolation during task execution

When Koen sends a Telegram message triggering a task, that message lands in the agent's **main Telegram session**. Everything the agent does inline — reading files, calling APIs, running shell commands — accumulates in that session's context window. For short conversational exchanges this is fine. For substantial implementation work (writing code, running builds, deploying), it pollutes the Telegram session with implementation noise and degrades context quality over time.

The correct pattern is for the agent to **spawn a sub-session** for the actual work and keep the Telegram session clean for orchestration and dialogue:

```
Koen message (Telegram session)
└ Agent understands the task
└ Agent spawns isolated sub-session via sessions_spawn
    └ Sub-session executes: code edits, builds, deploys,
      commits
        └ Sub-session reports result back to parent
└ Agent summarises outcome to Koen
└ Telegram session context: only the conversation, not the
  implementation
```

This is what the **coding-agent** skill wraps — it calls **sessions_spawn** under the hood, so the agent's Telegram session never fills up with tool call noise from the implementation work.

There is no Telegram slash command that forces sub-session spawning. The behaviour is governed by the agent's AGENTS.md instructions. If an agent is instructed to always delegate substantial implementation work via **sessions_spawn**, that happens automatically whenever a task is picked up — regardless of whether it was triggered by Telegram, an MC poll, or anything else.

What the agent does	Where it runs	Telegram session accumulates
Answers a question, reads a file	Inline in Telegram session	Yes
Executes a task (writes code, builds, deploys)	Spawned sub-session	No — only the summary
Cron-triggered task	Isolated cron session	Never touches Telegram session

Cross-agent coordination

Both OpenClaw and MC have native cross-agent features. IDEA uses them with a policy that matches the use case to the right mechanism.

Available mechanisms

OpenClaw — sessions_send tool (agent → agent, direct) Any agent can call **sessions_send** with the target agent's **sessionKey** to inject a message directly into that agent's running session. No Koen involvement.

The message lands in the target's session history and triggers a response. This is the OpenClaw-native agent-to-agent channel.

OpenClaw — `sessions_spawn` (delegate a bounded task) An agent can spawn an isolated sub-agent session, send it a specific task, and wait for the result. Useful for asking a peer agent a single bounded question (e.g. Atlas asking Axle "is this interface change backwards-compatible?") without opening an open-ended session.

MC — `POST /agent/gateway/boards/{board_id}/lead/message` (board-scoped message) Routes a message to a specific board lead via the gateway. Board-scoped, logged in MC. Intended for structured cross-agent handoffs.

MC — `POST /agent/boards/{board_id}/agents/{agent_id}/nudge` (Atlas → stalled agent) Atlas posts a targeted nudge to a specific agent. The message is injected into that agent's active session as a directive. Designed specifically for Atlas's quality manager role — Atlas detects a stalled or unresponsive agent and pokes it directly without Koen needing to relay.

MC — `POST /agent/gateway/leads/broadcast` (one → all) Atlas broadcasts a message to all board leads simultaneously. Fan-out pattern.

Policy: when to use each

Use case	Mechanism	Koen involved?
Atlas nudges a stalled agent	MC nudge endpoint	No
Atlas asks Axle a bounded technical question	<code>sessions_spawn</code> (isolated, awaits reply)	No
Atlas sends a review finding to Pixel	<code>sessions_send</code>	No
Atlas broadcasts a cross-cutting concern to all agents	MC broadcast	No
Agent needs a decision only Koen can make	Telegram message in own group	Yes
Task assignment / new work direction	Koen creates MC task	Yes
Agent proposes a design that affects another agent's repo	Post in own Telegram group, Koen decides	Yes

Rule

Agents use native cross-agent mechanisms freely for **operational coordination** (questions, review findings, nudges, bounded sub-tasks).

They route through Koen for **decisions** — anything that commits to a direction, assigns new work, or affects another agent's repo. The distinction: coordination is agent-to-agent; decisions require the CEO.

All cross-agent messages sent via OpenClaw tools are logged in session history. MC-routed messages are logged in MC. There is a full audit trail without Koen being the relay.

4. OpenClaw Configuration

Target `~/.openclaw/openclaw.json` structure (annotated)

```
{
  // Session behaviour
  session: {
    reset: {
      idleMinutes: 2160 // 36h idle → fresh session; daily reset
    },
    disabled: false
  },

  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6"
      },
      models: {
        "anthropic/claude-sonnet-4-6": {
          alias: "sonnet",
          params: { cacheRetention: "short" }
        }
      },
      contextPruning: {
        mode: "cache-ttl",
        ttl: "1h"
      },
      compaction: {
        mode: "safeguard"
      },
      heartbeat: {
        every: "0m" // Heartbeat disabled – cron handles MC
      },
      polling (see §6): false
    },
    list: [
```

```

    {
      id: "lead-6bddb9d2-c06f-444d-8b18-b517aeaa6aa8",
      name: "Axle",
      workspace: "/home/node/workspace/agents/agent-engine-dev",
      agentDir: "/home/pi/.openclaw/agents/lead-6bddb9d2-
c06f-444d-8b18-b517aeaa6aa8/agent"
    },
    {
      id: "lead-ac508766-e9e3-48a0-b6a5-54c6ffcdc1a3",
      name: "Pixel",
      workspace: "/home/node/workspace/agents/agent-console-
dev",
      agentDir: "/home/pi/.openclaw/agents/lead-ac508766-
e9e3-48a0-b6a5-54c6ffcdc1a3/agent"
    },
    {
      id: "lead-7cc2a1cf-fa22-485f-b842-bb22cb758257",
      name: "Beacon",
      workspace: "/home/node/workspace/agents/agent-site-dev",
      agentDir: "/home/pi/.openclaw/agents/lead-7cc2a1cf-
fa22-485f-b842-bb22cb758257/agent"
    },
    {
      id: "lead-3f1be9c8-87e7-4a5d-9d3b-99756c35e3a9",
      name: "Marco",
      workspace: "/home/node/workspace/agents/agent-programme-
manager",
      agentDir: "/home/pi/.openclaw/agents/
lead-3f1be9c8-87e7-4a5d-9d3b-99756c35e3a9/agent"
    },
    {
      id: "operations-manager",
      name: "Atlas",
      workspace: "/home/node/workspace/agents/agent-operations-
manager"
    }
  ]
  // NOTE: Remove the two mc-gateway-XXXX entries – these are
stale MC artefacts
],

bindings: [
  // Unchanged – one group per agent
  {
    agentId: "lead-6bddb9d2-c06f-444d-8b18-b517aeaa6aa8",
    match: { channel: "telegram", peer: { kind: "group", id:

```

```

"-5146184666" } }
  },
  {
    agentId: "lead-ac508766-e9e3-48a0-b6a5-54c6ffcdc1a3",
    match: { channel: "telegram", peer: { kind: "group", id:
"-5187034968" } }
  },
  {
    agentId: "lead-7cc2a1cf-fa22-485f-b842-bb22cb758257",
    match: { channel: "telegram", peer: { kind: "group", id:
"-5139661372" } }
  },
  {
    agentId: "lead-3f1be9c8-87e7-4a5d-9d3b-99756c35e3a9",
    match: { channel: "telegram", peer: { kind: "group", id:
"-5141459717" } }
  },
  {
    agentId: "operations-manager",
    match: { channel: "telegram", peer: { kind: "group", id:
"-5105695997" } }
  }
],

gateway: {
  mode: "local",
  bind: "lan", // Changed from "loopback" – MC Docker container
needs to reach gateway
  trustedProxies: ["127.0.0.1", "::1"],
  remote: { token: "<existing token>" },
  auth: { mode: "token", token: "<existing token>" },
  controlUi: {
    allowedOrigins: [
      "http://localhost:18789",
      "http://127.0.0.1:18789",
      "http://idea.tail2d60.ts.net:18789",
      "https://idea.tail2d60.ts.net:18789"
    ]
  }
},

channels: {
  defaults: {
    heartbeat: { showOk: false, showAlerts: true, useIndicator:
true }
  },

```

```

    telegram: {
      enabled: true,
      botToken: "<existing token>",
      dmPolicy: "allowlist",
      allowFrom: ["8320646468"],
      groupAllowFrom: ["8320646468"],
      groupPolicy: "allowlist",
      groups: { "*": { requireMention: false } },
      streamMode: "partial"
    }
  },

  commands: {
    native: "auto",
    nativeSkills: "auto",
    restart: true
  },

  skills: {
    load: {
      extraDirs: ["/home/node/workspace/skills"]
    }
  },

  plugins: {
    entries: {
      telegram: { enabled: true }
    }
  }
}

```

Key config changes from current state

Setting	Current	Target	Reason
<code>gateway.bind</code>	"loopback"	"lan"	MC Docker needs to reach gateway directly
<code>session.reset.idleMinutes</code>	not set (daily reset active)	2160	Disable daily reset; 36h idle window; manual / new for on-

Setting	Current	Target	Reason
			demand resets
<code>agents.defaults.heartbeat.every</code>	<code>"30m"</code>	<code>"0m"</code>	Cron handles MC polling; heartbeat disabled
<code>mc-gateway-XXXX</code> agents	2 entries	removed	Stale MC artefacts from provisioning

5. Installation — Clean Setup

The socat problem (and why it no longer exists)

The current setup routes MC→OpenClaw communication through Tailscale Serve, which strips WebSocket query parameters. The workaround was a `socat` proxy that re-adds those parameters before forwarding to the gateway.

This problem disappears when MC connects to OpenClaw directly.

MC and OpenClaw both run on the same Pi. The only reason to route through Tailscale Serve was the assumption that MC needed to reach OpenClaw via the external Tailscale hostname. It does not.

With `gateway.bind: "lan"`, the OpenClaw gateway listens on the Pi's LAN address (including the Docker bridge interface `172.17.0.1`). MC's Docker container can connect directly: `http://172.17.0.1:18789`.

No socat. No extra_hosts hacks. No custom WebSocket proxying.

What Tailscale Serve does in the clean setup

Tailscale Serve provides external HTTPS access for Koen's browser: - `https://idea.tail2d60.ts.net:18789` → OpenClaw Control UI + gateway API - `https://idea.tail2d60.ts.net:4000` → MC frontend - `https://idea.tail2d60.ts.net:8000` → MC backend API

Note: `gateway.bind: "lan"` makes the gateway listen on all LAN interfaces including the Tailscale interface, so Tailscale Serve can proxy it for browser access. MC connects to the gateway over the Docker bridge (`172.17.0.1:18789`), not through Tailscale.

Clean installation procedure (fresh Pi)

This is the target install for a fresh Pi, starting from the OpenClaw-documented path and adding only what IDEA needs on top.

Step 1 — Install OpenClaw (standard path)

```
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw onboard --install-daemon
```

The wizard configures: model provider + API key, gateway mode, basic channel setup. It installs OpenClaw as a systemd user service running as `pi`.

Step 2 — Symlink workspace path

Agents reference files at `/home/node/workspace/`. The workspace lives at `/home/pi/idea/`. A symlink makes both paths valid without touching agent config:

```
sudo mkdir -p /home/node
sudo ln -sf /home/pi/idea /home/node/workspace
```

Step 3 — Configure `~/.openclaw/openclaw.json`

Apply the target config from §4. Key settings: - `gateway.bind: "lan"` — allows MC Docker to connect - `session.reset.idleMinutes: 2160` — disable daily reset; 36h idle window - `agents.defaults.heartbeat.every: "0m"` — disable heartbeat polling - All 5 agent entries with correct workspace paths and agentDirs - All 5 bindings to correct Telegram group IDs

Apply with:

```
openclaw gateway restart
openclaw agents list --bindings    # verify bindings
openclaw gateway status           # verify running
```

Step 4 — Install Mission Control (standard path)

Use the official MC compose stack with published Docker images (not built from source):

```
git clone https://github.com/abhi1693/openclaw-mission-control.git
\
/home/pi/openclaw/mission-control
cd /home/pi/openclaw/mission-control
cp .env.example .env
# Edit .env: set LOCAL_AUTH_TOKEN (min 50 chars),
POSTGRES_PASSWORD, etc.
```

MC backend gateway URL — point to local OpenClaw, not Tailscale:

In `.env` (or `compose.yaml` environment section):

```
# The MC backend connects to OpenClaw directly over the Docker
bridge
GATEWAY_URL=http://172.17.0.1:18789
```

`172.17.0.1` is the Docker host bridge IP — routable from any Docker container to the host. OpenClaw listens there because `gateway.bind: "lan"` includes all LAN interfaces.

MC frontend — point to Tailscale HTTPS URL for browser access:

```
NEXT_PUBLIC_API_URL=https://idea.tail2d60.ts.net:8000
```

Start:

```
docker compose up -d
```

Step 5 — Tailscale Serve (external HTTPS access)

```
tailscale serve https:18789 localhost:18789 # OpenClaw Control UI
tailscale serve https:4000 localhost:4000  # MC frontend
tailscale serve https:8000 localhost:8000  # MC backend API
```

Verify:


```
tailscale serve status
curl https://idea.tail2d60.ts.net:18789 # should show OpenClaw
Control UI
curl https://idea.tail2d60.ts.net:4000 # should show MC frontend
```

Step 6 — Clone agent repos and org root

```
git clone https://github.com/koenswings/idea.git /home/pi/idea
cd /home/pi/idea/agents
git clone https://github.com/koenswings/agent-engine-dev.git
git clone https://github.com/koenswings/agent-console-dev.git
git clone https://github.com/koenswings/agent-site-dev.git
git clone https://github.com/koenswings/agent-programme-
manager.git
git clone https://github.com/koenswings/agent-operations-
manager.git
```

Step 7 — Install agent cron jobs for MC task pickup

For each agent, create an OpenClaw cron job (isolated agentTurn, every 15 minutes) that checks the MC board for new in_progress tasks. See §6 for details.

Step 8 — Install graphify cache and post-commit hooks

See §8.

Step 9 — Install backup cron

See §9.

6. Agent Trigger Model

Why heartbeats are disabled

Heartbeat wakes the full main session on a fixed interval regardless of whether there is anything to do. With 5 agents at 30-minute intervals, that is ~240 full model turns per day of agents checking "anything to do? No. OK." — significant credit burn with near-zero signal.

The replacement: isolated cron jobs for MC task polling

Each agent has one OpenClaw cron job. The job fires every 15 minutes, runs in an **isolated session** (not the main Telegram session), calls the MC API to check for **in_progress** tasks assigned to that agent, and either: - Finds nothing new → exits silently (no Telegram message, near-zero token cost) - Finds a new task → injects a system event into the agent's main session to pick it up

Cost profile: An isolated cron run that finds nothing and exits uses ~150-300 tokens. At 96 runs/day × 5 agents × 300 tokens = ~144k tokens/day maximum — but most runs exit immediately with minimal cost. Orders of magnitude cheaper than full heartbeat sessions that load full context.

Cron job definition per agent

The cron tool in the current system supports **sessionTarget: "isolated"** with **payload.kind: "agentTurn"** — a self-contained isolated run that receives a specific prompt. This is the OpenClaw-native mechanism for event-driven agent wakeup.

Cron job spec (create once per agent using the cron tool):

```
{
  "name": "<AgentName> MC task poll",
  "schedule": { "kind": "every", "everyMs": 900000 },
  "sessionTarget": "isolated",
  "payload": {
    "kind": "agentTurn",
    "message": "MC POLL: Check your Mission Control board for
tasks in 'in_progress' status. Use the mc-api skill. If you find a
task that you have not yet started working on, post a system event
to your main session notifying yourself to pick it up. If nothing
needs attention, exit silently with no reply.",
    "timeoutSeconds": 60
  },
  "delivery": { "mode": "none" }
}
```

Create these cron jobs using the **cron** tool with **action: "add"** and the job spec above, once per agent. Jobs persist in **~/.openclaw/cron/jobs.json** and survive gateway restarts.

Making pickup immediate when needed

For cases where Koen wants an agent to pick something up right now without waiting up to 15 minutes, the existing mechanism still works: send

a Telegram message to the agent's group. The message goes to the main session immediately, bypassing the cron poll cycle.

This is not a fallback — it is the intentional design. Cron polling handles async background pickup. Telegram handles urgent direction.

MC heartbeat status (agent online/offline indicator)

The MC online/offline indicator in the board UI requires agents to call `POST /api/v1/agents/{id}/heartbeat` explicitly. This is separate from OpenClaw heartbeats.

Decision: Wire it up. At the start of each Telegram-triggered session (not cron sessions), each agent posts a heartbeat to MC. This makes the board indicator meaningful: "online" means the agent recently responded to a real message.

Implementation: one line in AGENTS.md startup instructions per agent. See §11 for details.

7. Branch and Merge Workflow

No GitHub PRs — branches only

Agents work on feature branches. GitHub Pull Request objects are never created. The GitHub compare view (github.com/koenswings/<repo>/compare/main...<branch>) provides a diff view on demand. The MC task comment thread is the review record.

Why branches, not direct-to-main: - Isolates work-in-progress from the live codebase - Koen can see the complete diff before anything lands on main - A mistake is confined to a branch until merged - Allows agents to commit incrementally during work without polluting main

Why no GitHub PRs: - No PR ceremony (no PR number, no GitHub review UI, no PR comments) - The MC task IS the review unit — context lives in the task thread - No waiting for CI on a PR (GitHub Actions on push to main provides equivalent coverage)

The merge flow in detail

```
Agent completes work
→ All commits on feature branch (e.g. feat/copy-app-fix)
→ Moves MC task to "review"
→ Posts in task comment: "Work complete on branch feat/copy-app-fix.
Diff: github.com/koenswings/agent-engine-dev/compare/main...feat/copy-app-fix"
```

Koen reviews

- Reads task comment thread for context
- Optionally opens the GitHub compare URL for diff
- May post questions/feedback as task comments
- Agent responds in task comments, pushes fixes to same branch

Koen approves

- Moves task to "done" in MC
- This is the merge signal

Agent detects approval (next cron poll or Telegram message)

- Executes: `git checkout main && git merge --no-ff feat/copy-app-fix`
- Executes: `git push origin main`
- Posts confirmation in task comment
- Branch is now merged; graphify rebuild triggers (see §8)

Each agent merges their own code

No Atlas involvement in merges. Each agent owns their repo: - Axle owns and merges `agent-engine-dev` - Pixel owns and merges `agent-console-dev` - Beacon owns and merges `agent-site-dev` - Marco owns and merges `agent-programme-manager` - Atlas owns and merges `agent-operations-manager`

The `idea` org root is an exception — Atlas or Koen handles those merges since they affect all agents.

8. Graphify Integration

The full graphify design is documented in: `/home/pi/idea/agents/agent-operations-manager/design/graphify-integration.md`

This section documents the trigger mechanism that replaces the GitHub Actions approach in that doc, and how agents consume the graph.

Design decision: local trigger, not GitHub Actions

The `graphify-integration.md` doc proposes GitHub Actions as the rebuild trigger. **This design supersedes that choice** in favour of a local trigger:

Reason: With no-PR workflow, merges happen locally on the Pi (agents run `git merge && git push origin main`). A local post-commit hook fires immediately and reliably at merge time — no GitHub webhook latency, no Azure VM spinup cost, no need for Actions secrets. GitHub

Actions is the right choice for a team using GitHub-native merge workflows; it is unnecessary overhead when merges happen locally.

Trigger mechanism: post-commit hook + safety cron

Primary trigger — git post-commit hook in each agent repo:

Each agent repo gets an identical hook at `.git/hooks/post-commit`:

```
#!/bin/bash
# Trigger graphify rebuild when a commit lands on main
BRANCH=$(git rev-parse --abbrev-ref HEAD)
if [ "$BRANCH" = "main" ]; then
    nohup bash /home/pi/idea/scripts/rebuild-graph.sh \
        >> /home/pi/idea/logs/graphify-rebuild.log 2>&1 &
fi
```

```
chmod +x .git/hooks/post-commit
```

This fires immediately after every commit to main in that repo. `nohup` & backgrounds the rebuild so it does not block the agent's git operation.

Safety net — 5-minute hash-watching cron (system cron, pi user):

```
# /home/pi/idea/scripts/check-graph-stale.sh
#!/bin/bash
HASH_FILE=/home/pi/idea/logs/last-graphify-hash
CURRENT=$(git -C /home/pi/idea rev-parse HEAD 2>/dev/null)
LAST=$(cat "$HASH_FILE" 2>/dev/null)
if [ "$CURRENT" != "$LAST" ]; then
    echo "$CURRENT" > "$HASH_FILE"
    nohup bash /home/pi/idea/scripts/rebuild-graph.sh \
        >> /home/pi/idea/logs/graphify-rebuild.log 2>&1 &
fi
```

Crontab entry (pi user):

```
*/5 * * * * bash /home/pi/idea/scripts/check-graph-stale.sh
```

This catches any commits that bypassed the hook (manual pushes, CI, etc.) within 5 minutes.

The rebuild script

`/home/pi/idea/scripts/rebuild-graph.sh` — runs graphify across the full idea root:

```
#!/bin/bash
set -e
echo "[$(date)] Graphify rebuild triggered" >> /home/pi/idea/logs/graphify-rebuild.log
cd /home/pi/idea
source /home/pi/graphify-env/bin/activate
graphify . --update --no-viz --obsidian \
    >> /home/pi/idea/logs/graphify-rebuild.log 2>&1
echo "[$(date)] Graphify rebuild complete" >> /home/pi/idea/logs/graphify-rebuild.log
```

The rebuild reads the entire idea root (all agent repos, all shared docs), respecting `.graphifyignore`. Output lands at `/home/pi/idea/graphify-out/`.

Note: The rebuild runs on the main Pi, not a fleet Pi. This is acceptable because: - Graphify Pass 1 (AST parsing) is fast and CPU-light - Pass 3 (LLM extraction) only re-runs for changed files (SHA256 cache) — typically 0–2 files per merge - The rebuild runs in the background and does not block the gateway or any active session

How agents consume the graph

Every agent reads `/home/pi/idea/graphify-out/GRAPH_REPORT.md` at session start. This is already in each agent's AGENTS.md (added during the graphify integration phase). The file is always current (rebuilt within minutes of any push to main).

Agents do not need to be notified of graph updates. They read it fresh at every session start. Since sessions start fresh after 36 hours of idle or after a `/new`, agents naturally pick up graph changes within the idle window.

For long-running sessions doing significant cross-codebase work, agents should explicitly re-read the file at task start as a standing instruction in AGENTS.md.

Graph scope

One graph from the `idea/` root covering all repos. Cross-repo edges (Engine ↔ Console, code ↔ design docs, compose.yaml ↔ Engine config) emerge naturally. Per-repo graphs are not needed and not built.

9. Backup Strategy

What needs to be backed up

Item	Location	Risk if lost
Agent identity + memory files	<code>/home/pi/idea/agents/*/</code>	Medium — agent loses memory and identity
MC PostgreSQL database	Docker volume <code>mc-db-data</code>	High — all tasks, comments, board history lost
OpenClaw config	<code>~/.openclaw/openclaw.json</code>	Low — reconstructible, but tedious
OpenClaw agent state	<code>~/.openclaw/agents/*/</code>	Medium — sessions and auth profiles
Graphify output	<code>/home/pi/idea/graphify-out/</code>	Low — auto-rebuilds on next push

Backup destination

All backups push to the existing `agent-identities` GitHub repo (private). No new credentials needed — the nightly cron already has push access.

New directory structure:

```
agent-identities/  
  agents/           ← existing: identity + memory files per agent  
  backups/  
    mc-YYYYMMDD.dump      ← MC PostgreSQL dump (compressed  
    custom format)  
    openclaw-YYYYMMDD.tar.gz ← ~/.openclaw state
```

Backup cron script

`/home/pi/idea/scripts/backup-platform.sh:`

```

#!/bin/bash
set -e
DATE=$(date +%Y%m%d)
BACKUP_DIR=/home/pi/idea/backups
IDENTITIES_REPO=/home/pi/agent-identities

mkdir -p "$BACKUP_DIR"

# 1. MC PostgreSQL dump
PGPASSWORD=$(cat /home/pi/idea/platform/secrets/
mc_db_password.txt) \
    pg_dump -h 127.0.0.1 -p 5432 -U postgres mission_control \
    --format=custom > "$BACKUP_DIR/mc-$DATE.dump"

# 2. OpenClaw state (config + agent dirs)
tar czf "$BACKUP_DIR/openclaw-$DATE.tar.gz" \
    --exclude='~/.openclaw/agents/*/sessions/*.jsonl' \
    ~/.openclaw/

# 3. Copy to agent-identities repo and push
cp "$BACKUP_DIR/mc-$DATE.dump" "$IDENTITIES_REPO/backups/"
cp "$BACKUP_DIR/openclaw-$DATE.tar.gz" "$IDENTITIES_REPO/backups/"

# 4. Push
cd "$IDENTITIES_REPO"
git add backups/
git commit -m "backup: $DATE" --allow-empty
git push origin main

# 5. Prune local backups older than 7 days
find "$BACKUP_DIR" -name "*.dump" -mtime +7 -delete
find "$BACKUP_DIR" -name "*.tar.gz" -mtime +7 -delete

echo "[$(date)] Platform backup complete ($DATE)"

```

Note: session transcripts ([.jsonl](#) files) are excluded from the OpenClaw backup — they are large and easily regenerated. Only config, auth profiles, and agent metadata are backed up.

Backup schedule

Run at 03:30 UTC daily, 30 minutes after the identity backup:


```
30 3 * * * bash /home/pi/idea/scripts/backup-platform.sh >> /home/
pi/idea/logs/backup.log 2>&1
```

Restore procedure

After a fresh install (\$5), with MC running:

```
# 1. Restore MC database
PGPASSWORD=$(cat /home/pi/idea/platform/secrets/
mc_db_password.txt) \
    pg_restore -h 127.0.0.1 -p 5432 -U postgres -d mission_control \
    /path/to/mc-YYYYMMDD.dump

# 2. Restore OpenClaw state
tar xzf /path/to/openclaw-YYYYMMDD.tar.gz -C /

# 3. Re-link MC to new OpenClaw gateway token
# POST to MC API: /api/v1/gateways/<gateway_id>/templates/sync?
rotate_tokens=true
# (After gateway is running and MC can connect to it)

# 4. Restart OpenClaw
openclaw gateway restart
```

10. Skills Inventory

Shared skills (</home/node/workspace/skills/>)

Skill	Keep?	Reason
mc-api	Keep + update	Core to the new workflow; becomes more heavily used
md-to-pdf	Keep	No dependency on removed workflow
telegram-table	Keep	Still on Telegram; tables still a problem

Built-in OpenClaw skills

Skill	Keep?	Reason
clawflow	Keep	

Skill	Keep?	Reason
		Background orchestration substrate; relevant for cron flows
coding-agent	Keep	Delegate to Codex/Claude Code for deep coding tasks
github	⚠ Reduce scope	No PRs; keep for gh CLI operations (issues, CI checks)
healthcheck	Keep	Security auditing, unaffected
node-connect	Keep	Node pairing diagnostics, unaffected
skill-creator	Keep	Creating and improving skills
gh-issues	Drop	Entire purpose is PR workflow — no longer applicable
tmux	Drop	Not used since OpenClaw native install replaced Tabby/SSH-based workflow
weather	Drop	Not relevant to IDEA operations

mc-api skill — update required

The mc-api skill needs to be updated to reflect: - The new work cycle (task polling, branch names in comments, merge detection) - The MC heartbeat call at session start - Removal of BACKLOG.md export references

Atlas is responsible for this update as part of the migration.

11. MC Heartbeat — Online/Offline Status

What it does

The MC board shows each agent as "online" or "offline". This is driven by the agent calling `POST /api/v1/agents/{id}/heartbeat` on the MC API. It has no effect on agent behaviour — it is a visibility indicator for Koen.

Decision: Wire it up. Cost is one API call at session start. Value is that the MC board gives an honest health signal.

Implementation

Add to each agent's AGENTS.md startup section:

At the start of each Telegram-triggered session (not cron sessions):

1. Call POST /api/v1/agents/{mc_agent_id}/heartbeat on the MC API (Use the mc-api skill; credentials in .env as AUTH_TOKEN)

Each agent needs to know their own MC agent ID. This is distinct from the OpenClaw agent ID. It is set during MC board provisioning and stored in each agent's TOOLS.md.

Cron sessions must NOT post a heartbeat. Cron runs are isolated and frequent; posting a heartbeat from them would make the indicator permanently "online" even when the agent is not actually responding to real messages.

12. What Changes vs Current Setup

Removed

Item	Reason
BACKLOG.md export script + cron	MC board is the live source; no file mirror needed
<code>gateway.bind: "loopback"</code>	Replaced by " <code>lan</code> " to enable direct MC connection
socat proxy service	Not needed when MC connects directly to OpenClaw
Daily session reset (4 AM)	Replaced by 36-hour idle reset; manual /new for on-demand resets
<code>agents.defaults.heartbeat.every: "30m"</code>	Replaced by isolated cron job polling
2× <code>mc-gateway-XXXX</code> agents in <code>openclaw.json</code>	Stale MC provisioning artefacts
GitHub PR workflow	Replaced by branch + MC task approval + local merge
<code>gh-issues</code> , <code>tmux</code> , <code>weather</code> skills	No longer applicable

Added

Item	Description
<code>session.reset.idleMinutes: 2160</code>	36-hour idle session reset
MC polling cron jobs (5×)	One per agent, every 15 min, isolated runs
MC heartbeat call at session start	Per agent, Telegram sessions only
Post-commit hook in each agent repo	Triggers graphify rebuild on merge to main
Hash-watching cron	Safety net for graphify rebuild
Platform backup cron	Daily pg_dump + OpenClaw state → agent-identities repo

Unchanged

Item	Status
Telegram group bindings	Same group IDs, same 1:1 agent:group model
Agent workspace paths	Same (symlink at /home/node/workspace preserved)
MC Docker stack	Same services, same port bindings
Tailscale Serve config	Same (MC frontend + backend via HTTPS)
Agent repos and GitHub remotes	Same
Cross-agent relay via Koen	Replaced by native OpenClaw/MC mechanisms for coordination; Koen only for decisions
Skills: mc-api, md-to-pdf, telegram-table	Same, mc-api needs content update
Identity files: AGENTS.md, SOUL.md, etc.	Same structure; startup checklists updated

13. Migration Runbook

This runbook migrates the live system from current state to target state. Steps are ordered to minimise downtime and are individually reversible.

Prerequisite: Read this entire document before starting. Step 2 causes ~60 seconds of OpenClaw downtime. Everything else is non-disruptive.

Who executes this migration

This migration must be executed by a Claude Code session on the Pi, not from witside an OpenClaw agent session.

Reason: Step 2 restarts the OpenClaw gateway. Any OpenClaw agent session — including Axle, Atlas, or any other — is killed by that restart. An agent that starts this process cannot complete it. This is the same constraint that applied to the Docker → native migration in April 2026.

Claude Code (`claude`) runs as an independent process directly on the Pi as the `pi` user. It is not affected by OpenClaw restarting, Docker containers restarting, or anything else on the machine.

Pre-migration: snapshot current state (rollback anchor)

Execute this entire block before touching anything else — before Step 0, before starting Claude Code. It creates a complete, named rollback point across every repo and config file. If the migration goes wrong at any step, the rollback procedure below restores exactly this state.

1. Commit any uncommitted changes in every repo

Some repos have uncommitted changes (AGENTS.md edits, memory files, etc.). These must be committed before tagging so the tag captures a clean, complete snapshot. For each repo with changes, review `git status` first — do not blindly `git add -A` as some repos contain untracked files that must NOT be committed (`.openclaw/`, `node_modules/`, zip files). Commit only intentional changes:

```
# For each repo that has changes to commit:
cd /home/pi/idea/<repo>
git status                # review what changed
git add <specific files>  # add only what should be committed
git commit -m "chore: pre-migration snapshot"
git push origin HEAD
```

Repos to check: `idea` (org root), `agent-engine-dev`, `agent-console-dev`, `agent-site-dev`, `agent-programme-manager`, `agent-operations-manager`.

2. Tag every repo with **pre-mc-native-migration**

```
TAG="pre-mc-native-migration"
DATE=$(date +%Y-%m-%d)

for repo in /home/pi/idea \
            /home/pi/idea/agents/agent-engine-dev \
            /home/pi/idea/agents/agent-console-dev \
            /home/pi/idea/agents/agent-site-dev \
            /home/pi/idea/agents/agent-programme-manager \
            /home/pi/idea/agents/agent-operations-manager; do
    echo "Tagging $repo"
    git -C "$repo" tag -a "$TAG" -m "State before mc-native-
migration ($DATE)"
    git -C "$repo" push origin "$TAG"
done
```

Verify every tag landed on GitHub:

```
TAG="pre-mc-native-migration"
for repo in /home/pi/idea \
            /home/pi/idea/agents/agent-engine-dev \
            /home/pi/idea/agents/agent-console-dev \
            /home/pi/idea/agents/agent-site-dev \
            /home/pi/idea/agents/agent-programme-manager \
            /home/pi/idea/agents/agent-operations-manager; do
    echo -n "$(basename $repo): "
    git -C "$repo" ls-remote --tags origin "$TAG" | awk '{print $2}'
    || echo "MISSING"
done
```

Each line must print **refs/tags/pre-mc-native-migration**. Do not proceed if any tag is missing.

3. Back up OpenClaw config and state

```
mkdir -p /home/pi/backups
TIMESTAMP=$(date +%Y%m%d-%H%M)

# Full ~/.openclaw directory
tar czf /home/pi/backups/openclaw-pre-migration-$
```

```
{TIMESTAMP}.tar.gz ~/.openclaw/

# openclaw.json separately for quick reference during rollback
cp ~/.openclaw/openclaw.json /home/pi/backups/openclaw-pre-
migration-${TIMESTAMP}.json

echo "OpenClaw backup: $(ls -lh /home/pi/backups/openclaw-pre-
migration-${TIMESTAMP}.tar.gz)"
```

4. Back up MC database

```
mkdir -p /home/pi/backups
TIMESTAMP=$(date +%Y%m%d-%H%M)
MC_PW=$(cat /home/pi/idea/platform/secrets/mc_db_password.txt)

PGPASSWORD="$MC_PW" pg_dump \
  -h 127.0.0.1 -p 5432 -U postgres mission_control \
  --format=custom \
  > /home/pi/backups/mc-pre-migration-${TIMESTAMP}.dump

echo "MC backup: $(ls -lh /home/pi/backups/mc-pre-migration-$
{TIMESTAMP}.dump)"
```

5. Back up compose.yaml

```
TIMESTAMP=$(date +%Y%m%d-%H%M)
cp /home/pi/idea/platform/compose.yaml \
  /home/pi/backups/compose-pre-migration-${TIMESTAMP}.yaml
echo "compose.yaml backed up"
```

After all five steps: confirm `/home/pi/backups/` contains three timestamped files and every repo has the tag on GitHub. Only then proceed to Step 0.

Rollback procedure

If the migration goes wrong at any point, restore pre-migration state:

1. OpenClaw config:

```

openclaw gateway stop

# Replace TIMESTAMP with the value from the pre-migration backup
filename
cp /home/pi/backups/openclaw-pre-migration-<TIMESTAMP>.json \
  ~/.openclaw/openclaw.json

openclaw gateway start

```

2. MC Docker stack:

```

# Restore compose.yaml
cp /home/pi/backups/compose-pre-migration-<TIMESTAMP>.yaml \
  /home/pi/idea/platform/compose.yaml

# Restore MC database (only if DB was changed during migration)
MC_PW=$(cat /home/pi/idea/platform/secrets/mc_db_password.txt)
PGPASSWORD="$MC_PW" pg_restore \
  -h 127.0.0.1 -p 5432 -U postgres -d mission_control --clean \
  /home/pi/backups/mc-pre-migration-<TIMESTAMP>.dump

# Restart stack
cd /home/pi/idea/platform && docker compose down && docker compose
up -d

```

3. Agent repos (if any files were changed during migration):

```

TAG="pre-mc-native-migration"
for repo in /home/pi/idea \
  /home/pi/idea/agents/agent-engine-dev \
  /home/pi/idea/agents/agent-console-dev \
  /home/pi/idea/agents/agent-site-dev \
  /home/pi/idea/agents/agent-programme-manager \
  /home/pi/idea/agents/agent-operations-manager; do
  git -C "$repo" checkout "$TAG"
done

```

4. Tailscale rename: Go to the Tailscale admin console and rename **idea** back to **openclaw-pi**.

5. Verify: Send a Telegram message to any agent. Check MC board loads.

Step 0 — Rename Tailscale machine to `idea` (no downtime)

This renames the service identity so all URLs use `idea.tail2d60.ts.net` and survive hardware changes (if the platform moves to a new Pi, rename the new machine to `idea` and all URLs remain stable).

1. Open the Tailscale admin console: <https://login.tailscale.com/admin/machines>
2. Find the `openclaw-pi` machine entry
3. Click the three-dot menu → **Edit machine name** → change to `idea`
4. Save

MagicDNS propagates within seconds. Old `openclaw-pi.tail2d60.ts.net` URLs stop working immediately — update anything that references them:

- `~/.openclaw/openclaw.json` → `gateway.controlUi.allowedOrigins`
- `platform/compose.yaml` → `CORS_ORIGINS` and `NEXT_PUBLIC_API_URL`
- Any browser bookmarks

Verify:

```
curl https://idea.tail2d60.ts.net:18789 # OpenClaw Control UI
```

Starting the Claude Code session

SSH into the Pi and start a named tmux session:

```
ssh pi@<pi-ip-or-current-tailscale-address>
tmux new-session -s migration
cd /home/pi/idea
claude
```

The `tmux` session is essential — it survives SSH disconnects. If your connection drops mid-migration, reconnect and run `tmux attach -t migration` to resume exactly where you left off with Claude Code still running.

Opening prompt for Claude Code

Paste this as the first message when `claude` starts:

Read the following files before doing anything:

1. `/home/pi/idea/design/mc-native-platform-design.md` – the full migration design and runbook
2. `~/.openclaw/openclaw.json` – the current live OpenClaw config
3. `/home/pi/idea/platform/compose.yaml` – the current MC Docker stack
4. `/home/pi/idea/platform/.env` – MC credentials (do not log or expose these)

You are executing the migration runbook in Section 13, starting from Step 1.

The pre-migration snapshot and Step 0 (Tailscale rename) are already done.

Rules:

- Execute one step at a time. Verify each step worked before proceeding.
- If a step fails or produces an unexpected result, stop and report. Do not try to work around failures silently.
- The OpenClaw gateway will restart during Step 2. This is expected and will not affect you – you are running outside OpenClaw as a separate process.
- Do not touch anything not listed in the runbook.
- After each step, confirm what you did and what the verification showed.

Start with Step 1.

What Claude Code can do on this Pi

Running as `pi` it has access to: - Edit `~/.openclaw/openclaw.json` and run `openclaw gateway restart` - Edit `platform/compose.yaml` and run `docker compose` commands - Create scripts and crontab entries - Run `git` operations in any repo under `/home/pi/idea/`

It cannot access GitHub beyond what the Pi's SSH key allows — but nothing in this migration requires that.

Step 1 — Remove stale mc-gateway agents from openclaw.json (no downtime)

Edit `~/.openclaw/openclaw.json`. Remove the two entries with IDs: - `mc-gateway-5632ed12-add5-4935-8a19-9fff6d9a9c91` - `mc-gateway-ccc8463b-8b7d-46a1-afea-f699081b44c2`

These are stale artefacts from an earlier MC provisioning run. They have no bindings and serve no purpose.

OpenClaw hot-reloads config — no restart needed.

Verify: `openclaw agents list` — should show 5 agents only.

Step 2 — Change gateway.bind from loopback to lan (~60s downtime)

Edit `~/.openclaw/openclaw.json`:

```
"gateway": {  
  "bind": "lan",    // was "loopback"  
  ...  
}
```

Restart:

```
openclaw gateway restart
```

Verify gateway is listening on all interfaces:

```
ss -tlnp | grep 18789  
# Should show 0.0.0.0:18789, not 127.0.0.1:18789
```

Verify MC can connect to gateway:

```
# From inside the MC backend container:  
docker exec openclaw-mission-control-backend-1 \  
  curl -s http://172.17.0.1:18789/health
```

Step 3 — Configure session idle reset (no downtime)

Add to `~/.openclaw/openclaw.json`:

```
"session": {  
  "reset": {  
    "idleMinutes": 2160  
  }  
}
```

OpenClaw hot-reloads. No restart needed. Existing sessions are not affected immediately — the idle timer applies from the last real user interaction.

Step 4 — Disable heartbeat (no downtime)

Already set to `"0m"` in the default section of the current config. Verify:

```
grep -A2 '"heartbeat"' ~/.openclaw/openclaw.json
```

If not `"0m"`, set it and hot-reload.

Step 5 — Update MC compose to connect directly to gateway

In `/home/pi/idea/platform/compose.yaml`, find the `mission-control-backend` service and update (or add) the gateway URL environment variable:

```
environment:  
  - GATEWAY_URL=http://172.17.0.1:18789  
  # Remove any extra_hosts lines pointing Tailscale hostname to IP  
  # Remove any socat-related config
```

Remove the socat proxy service from `compose.yaml` if present.

Apply:

```
cd /home/pi/idea/platform
docker compose up -d --force-recreate mission-control-backend
mission-control-webhook-worker
```

Verify MC can reach the gateway via the MC UI (board should show gateway connected).

Step 6 — Create MC task polling cron jobs

For each of the 5 agents, create an isolated cron job using the cron tool. See §6 for the job spec. Use `cron` tool with `action: "add"`.

Verify: `openclaw cron list` — should show 5 new jobs.

Step 7 — Wire MC heartbeat call into each agent's AGENTS.md

For each agent, add the heartbeat call to the startup section. Find each agent's MC agent ID first (from the MC API or MC board settings), store it in the agent's TOOLS.md.

Add to each AGENTS.md:

```
At the start of each Telegram-triggered session, call POST /api/
v1/agents/{mc_agent_id}/heartbeat
using the mc-api skill. Do not call this from cron sessions.
```

Step 8 — Install post-commit hooks in each agent repo

For each agent repo:

```
cat > /home/pi/idea/agents/<repo>/.git/hooks/post-commit << 'EOF'
#!/bin/bash
BRANCH=$(git rev-parse --abbrev-ref HEAD)
if [ "$BRANCH" = "main" ]; then
    nohup bash /home/pi/idea/scripts/rebuild-graph.sh \
        >> /home/pi/idea/logs/graphify-rebuild.log 2>&1 &
fi
EOF
chmod +x /home/pi/idea/agents/<repo>/.git/hooks/post-commit
```

Also add for the org root (`/home/pi/idea/.git/hooks/post-commit`).

Step 9 — Install hash-watching cron

Create `/home/pi/idea/scripts/check-graph-stale.sh` (content in §8).

Add to pi user crontab:

```
*/5 * * * * bash /home/pi/idea/scripts/check-graph-stale.sh
```

Step 10 — Install backup cron

Create `/home/pi/idea/scripts/backup-platform.sh` (content in §9).

Add to pi user crontab:

```
30 3 * * * bash /home/pi/idea/scripts/backup-platform.sh >> /home/pi/idea/logs/backup.log 2>&1
```

Run once manually to verify it works:

```
bash /home/pi/idea/scripts/backup-platform.sh  
# Check agent-identities repo on GitHub for new backup/ files
```

Step 11 — Update mc-api skill

Atlas updates the mc-api skill content to reflect the new workflow: - Task polling pattern - Branch name in task comments - Merge detection on approval - MC heartbeat call - Remove BACKLOG.md export references

This is an Atlas task, created in MC.

Step 12 — Decommission removed components

Remove from the project (via PRs to the idea repo): - `scripts/export-backlog.sh` — BACKLOG.md export (no longer needed) - `scripts/check-new-tasks.sh` — old cross-agent task script (inactive, now replaced) - `BACKLOG.md` — auto-exported file (no longer generated) - Any socat service definition in `compose.yaml`

After the migration completes

1. Open a fresh Telegram message to any agent — verify the session comes up
 2. Check the MC board shows the gateway as connected
 3. Verify backup runs clean: `bash /home/pi/idea/scripts/backup-platform.sh`
 4. Exit Claude Code and close the tmux session: `exit` twice
-

Appendix: design decisions index

Decision	Section	Rationale
MC + Telegram as dual surfaces, different purposes	§2	Neither replicates the other
Idle reset (36h) instead of daily reset	§1, §4	Context continuity more valuable than forced freshness; manual / new gives on-demand control
Heartbeat disabled; isolated cron for MC polling	§6	Cost: targeted polling << full heartbeat sessions
Branch-first, no GitHub PRs, local merge on MC approval	§7	Safety of isolation without PR ceremony
Each agent merges their own repo; Atlas not involved	§7	Clean ownership; no Atlas as bottleneck
gateway.bind: "lan"; MC connects directly	§4, §5	Eliminates socat; clean standard architecture
Local post-commit hook triggers graphify; no GitHub Actions	§8	No-PR workflow means merges are local; local hooks are simpler
Single graph from idea/root	§8	Cross-repo edges emerge naturally; one graph simpler than 5
Backup to agent-identities repo, backups/subdir	§9	No new credentials; same push path as identity backup
MC online/offline indicator wired up	§11	Meaningful health signal; minimal cost
	§10	

Decision	Section	Rationale
gh-issues, tmux, weather skills dropped		No longer applicable to IDEA operations